Python Engineering - Reference Guide

Class 08 - Data Processing con NumPy

Guia rapida de mascaras booleanas, filtros, estadistica y operaciones habituales de procesado de datos.

1. Mascara booleana

```
mask = heights > 100

# Example
# [False, False, True, True, ...]
```

Una mascara tiene True donde la condicion se cumple y False donde no.

2. Filtrar

```
selected_values = heights[mask]
selected_rows = trajectory_array[mask]
```

3. Combinar condiciones

Operacion	NumPy	Ejemplo
AND	&	(h > 50) & (v > 0)
OR	|	(h > 100) | (v < -40)
NOT	~	~(v > 0)

Cada condicion debe ir normalmente entre parentesis.

4. Contar True

```
count = np.sum(mask)
# True behaves like 1, False like 0
```

5. np.where()

```
indices = np.where(mask)[0]
```

Devuelve indices donde la condicion es True. En un array 1D solemos tomar [0].

6. Estadistica basica

Funcion	Significado
np.mean(x)	media
np.std(x)	desviacion estandar
np.min(x)	minimo
np.max(x)	maximo
np.median(x)	mediana
np.sum(x)	suma

7. Diferencias consecutivas

```
dx = np.diff(x)
# dx[i] = x[i+1] - x[i]
```

La longitud pasa de N a N-1.

8. Anadir columnas

```
processed = np.column_stack((
    heights,
    velocities,
    times,
    energy
))
```

np.column_stack agrupa vectores 1D como columnas de una matriz 2D.

9. Seleccion por rango

```
mask = (times >= 2.0) & (times <= 5.0)
window = trajectory_array[mask]
```

10. Comprobar valores aproximadamente iguales

```
np.allclose(dt_samples, config['dt'])
```

Para floats, comparar con tolerancia suele ser mejor que exigir igualdad exacta con ==.

11. Patron mental

```
QUESTION
   -> CONDITION
      -> BOOLEAN MASK
         -> FILTER / COUNT / INDICES
            -> STATISTIC / ENGINEERING RESULT
```

Mini cheat sheet

```
mask = x > threshold
x[mask]                      filter values
A[mask]                      filter rows
np.sum(mask)                 count True
np.where(mask)[0]            indices
(c1) & (c2)                  AND
(c1) | (c2)                  OR
~c1                          NOT
np.mean(x), np.std(x)         stats
np.diff(x)                   consecutive differences
np.column_stack((a,b,c))      add columns
np.allclose(a,b)              approximate equality
```